

Entregable 1.

- **Asignatura:** Fundamentos de Arquitectura de Software
- **Caso de Estudio:** Sistema Digital de Juego de Cartas (Variante Clásica & Variante Apuntado a 101)
- **Marco Teórico Base:** *Fundamentals of Software Architecture: A Modern Engineering Approach* (2.^a edición, 2025) – Mark Richards & Neal Ford.

Integrantes del Equipo:

Nombre Completo	Doc. Identificación	Correo Electrónico
Angelika Moya Bolaños	1061702724	amoyab@unal.edu.co
Daniel Restrepo Faciolince	1033266333	darestrepof@unal.edu.co
Juan Manuel Teheran	1073968534	jteheranm@unal.edu.co
Thomas Monnier Granda	1038866661	tmonnier@unal.edu.co

Repositorio de Github: <https://github.com/Thom204/ApuntadoFdA>

Entregable 1. Guía de Especificación De Proyecto.....	1
1. Contexto de Negocio y Transición de Procesos.....	3
1.1 Modelo As-Is (Proceso Actual Manual).....	3
1.2 Modelo To-Be (Situación Futura Deseada).....	4
1.3 Matriz de Análisis de Brecha (Gap Analysis).....	5
1.4 Diagramas BPMN.....	6
2. Elicitación de Requerimientos y Casos de Uso.....	9
2.1 Especificación de Requerimientos y Reglas:.....	9
2.1.1 Requisitos Funcionales.....	9
2.1.2 Reglas de Dominio.....	11
2.2 Diagrama UML de Casos de Uso:.....	12
2.3 Fichas Descriptivas de Casos de Uso Críticos.....	14
3. Drivers Arquitectónicos.....	16
3.1 Tabla Resumen de Drivers Arquitectónicos.....	16
3.2 Análisis Detallado de los Atributos de Calidad Críticos.....	17
1. Configurabilidad y Extensibilidad (DA-01).....	17
2. Testabilidad (DA-02).....	17
3. Rendimiento y Responsiveness (DA-03).....	17
3.3 Gestión de Trade-offs (Compromisos Arquitectónicos).....	18
4. Arquitectura Lógica y Descomposición Modular.....	18
4.1 Identificación de Componentes Lógicos.....	18
4.1.1 Enfoque aplicado (Workflow).....	18
4.1.2 Mapeo Workflow → Componentes.....	18
4.2 Declaraciones de Rol y Responsabilidad.....	19
4.2.1 Declaraciones por componente.....	19
4.2.2 Justificación por características arquitectónicas.....	21
4.2.3 Verificación contra el Entity Trap.....	21
4.3 Diagrama UML de Componentes Lógicos.....	21
4.3.1 Diagrama.....	21
4.3.2 Lectura del diagrama (puertos y adaptadores, estereotipos).....	22
4.4 Diagrama UML de Clases de Dominio.....	22
4.4.1 Diagrama.....	23
4.4.2 Patrones de diseño e invariantes reflejados en el modelo.....	23
4.5 Evaluación de Acoplamiento y Ley de Deméter.....	24
4.5.1 Aplicación de la Ley de Deméter.....	24
4.5.2 Hallazgos y ajustes al modelo (H1–H5).....	25
4.5.3 Matriz de dependencias y métricas de acoplamiento (Ca, Ce, I).....	25
5. Decisiones Arquitectónicas Iniciales (ADRs).....	27
5.1 ADR-01: Particionar el sistema por dominio del juego mediante un monolito modular.	
5.2 ADR-02: Implementar las reglas de victoria como estrategias intercambiables dentro del monolito modular.....	28

1. Contexto de Negocio y Transición de Procesos

1.1 Modelo As-Is (Proceso Actual Manual)

En la situación actual, el juego de cartas se ejecuta de forma física e informal en una mesa compartida limitada a un grupo cerrado de **exactamente 5 jugadores**. El flujo operativo se compone de las siguientes fases manuales:

- **Barajado, Reparto y Escalabilidad Física:** Un jugador designado baraja físicamente la baraja tradicional (empleando múltiples mazos debido al volumen de participantes: 5 jugadores con manos de 10 o más cartas iniciales exigen al menos dos barajas estándar de 52 cartas más comodines). El reparto de cartas boca abajo se realiza de forma manual y secuencial en la mesa, dejando el excedente como mazo central y descubriendo la primera carta para iniciar el pozo de descarte.
- **Ciclo de Turnos Secuenciales en Grupo Amplio:** Los jugadores participan en turnos en sentido antihorario.
- **Validación Visual de Cierre (Cantada):** Cuando un jugador de la mesa considera que cumple con los requisitos de victoria de la modalidad activa —la **Modalidad Clásica (4-3-3)** o la **Modalidad por Apuntado** (tres ternas y sobrante < 5 puntos)—, detiene la partida. La validación de las combinaciones depende de que los otros jugadores verifiquen visualmente y de forma simultánea las cartas expuestas, determinando si en efecto es ganador de la ronda.
- **Conteo Colectivo y Anotación Manual:** Tras el cierre, los otros jugadores exponen sus cartas sobrantes. El anotador de la mesa calcula manualmente la suma aritmética de los puntajes de los 4 oponentes. Este acumulado se registra en una tabla manual.
- **Cierre de partida:** La partida repite los anteriores pasos cada que termina una ronda. Hay varias formas de cerrar la partida. Si un jugador alcanza o supera el umbral de los **-x puntos**, será eliminado y no podrá seguir jugando en la siguiente ronda, siempre que no quede un único jugador, en cuyo caso sería el ganador de la ronda y se le suman 10 puntos a su puntaje. La partida también puede terminar si un jugador tiene un puntaje mayor o igual a 51 puntos, caso en que dicho jugador sería ganador de la partida.

Puntos de Dolor Identificados para 5 Jugadores:

1. **Saturación y Retrasos en Mesa Grande:** Con 5 jugadores simultáneos, el flujo se vuelve lento y propenso a interrupciones por turnos perdidos o desacuerdos en el orden de juego.
2. **Alta complejidad en la Validación Visual Colectiva:** Verificar cartas, escalas y ternas de 5 personas simultáneamente a simple vista genera constantes disputas sobre la legitimidad de las jugadas.
3. **Colapso del Conteo Manual:** Sumar las cartas sobrantes de 4 perdedores ronda tras ronda aumenta drásticamente los errores aritméticos del anotador en físico.

4. **Descoordinación por Ausencias o Distracciones:** La falta de un orquestador automatizado en un grupo de 5 personas dificulta retomar la partida si un jugador se ausenta temporalmente.

1.2 Modelo To-Be (Situación Futura Deseada)

El modelo futuro sistematiza y automatiza el juego mediante una **aplicación web centralizada orientada a salas cerradas de exactamente 5 jugadores**, soportada por un monolito modular con particionamiento por dominio. El proceso digital opera de la siguiente manera:

- **Creación de Sala Web para 5 Jugadores:** Un usuario anfitrión crea una sala digital y configura la partida seleccionando la modalidad (Clásica o Apuntado), el límite de eliminación (-101 puntos por defecto), y restricciones de capacidad fijadas estrictamente en un **máximo y mínimo de 5 jugadores** por sesión. El sistema bloquea el inicio de la partida hasta que los 5 cupos están cubiertos y conectados.
- **Gestión Automatizada de Concurrencia y Turnos de 5 Actores:** El sistema inicializa la baraja compuesta (calculada para soportar las manos y descartes de 5 jugadores) mediante identificadores únicos de instancia. El componente *TurnOrchestrator* administra rigurosamente el vector circular de turnos entre los 5 participantes de la sala web, garantizando que solo el jugador activo pueda interactuar con el mazo o el pozo y aplicando un límite de tiempo por turno si se requiere.
- **Evaluación Algorítmica Paralela y Cierre en Red:** Al declarar el cierre, el *HandEvaluator* procesa instantáneamente la mano del jugador activo y contrasta las combinaciones bajo la regla seleccionada (Clasica o Apuntado < 5 pts).
- **Liquidación Simultánea y Actualización del Estado:** Confirmado el cierre válido, el componente *RoundScoringComponent* calcula en milisegundos las penalizaciones correspondientes a las cartas sueltas de los demás jugadores y suma 10 puntos al jugador ganador de la ronda. El estado actualizado del marcador global persiste en el repositorio de la partida y se transmite en tiempo real a los clientes conectados.
- **Detección Automática de Fin de Partida:** Tras cada liquidación, el sistema evalúa si algún participante ha alcanzado o superado el límite de -101 puntos acumulados. Al cumplirse esta condición, el jugador con dicho puntaje es eliminado, imposibilitando que siga jugando. El juego acaba en tres posibles escenarios.
 - *Solo queda un jugador*, en dicho caso éste sería el ganador de la partida.
 - *Un jugador alcanza 51 puntos*, por lo que se declara como ganador.

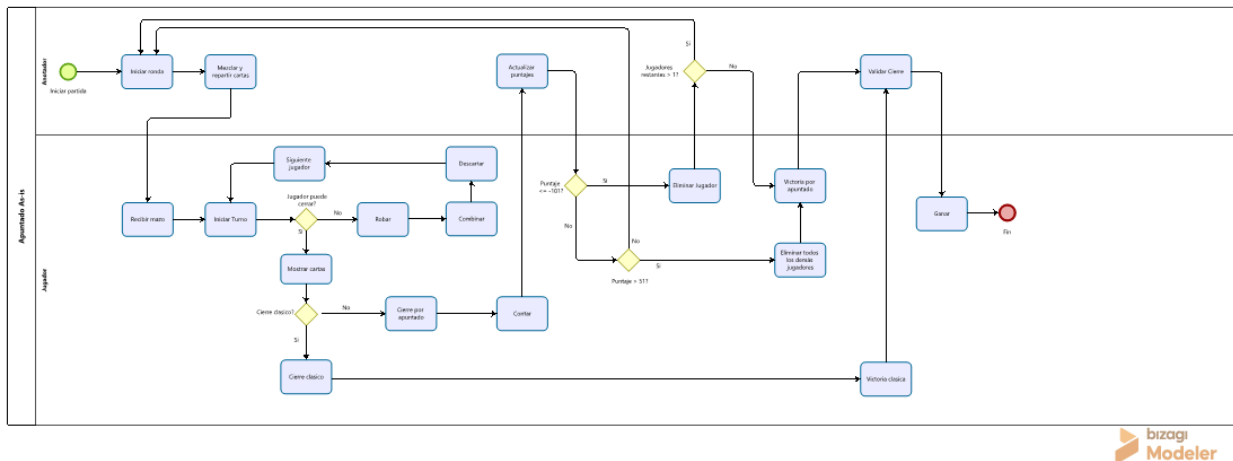
1.3 Matriz de Análisis de Brecha (Gap Analysis)

Dimensión	Estado Actual (As-Is)	Estado Deseado (To-Be)	Brecha Identificada (Gap)	Acción Arquitectónica Requerida
Procesos / Reglas	Mesa física limitada a 5 personas con validación visual subjetiva y turnos propensos a interrupciones.	Aplicación web con control estricto de salas de exactamente 5 jugadores y validación algorítmica instantánea.	Inexistencia de un motor de reglas concurrente y de un orquestador de estados para mesas cerradas de 5 usuarios.	Diseñar un componente <i>TurnOrchestrator</i> circular para 5 actores y un <i>HandEvaluator</i> desacoplado mediante el patrón <i>Strategy</i> .
Datos / Estado	Puntuaciones y turnos de 5 jugadores anotados manualmente en papel; alta vulnerabilidad a pérdida de información.	Persistencia estructurada en base de datos relacional de la sala, las manos privadas de los 5 jugadores, el historial de rondas y el marcador global.	Falta de trazabilidad transaccional y de un modelo de datos capaz de aislar las manos privadas de 5 jugadores concurrentes.	Modelar entidades relacionales normalizadas (Partida, Sala, 5 Jugadores por Sala, Rondas, Movimientos) con control de sesiones web.
Tecnología / UX	Interacción presencial física de 5 personas alrededor de una mesa.	Aplicación web multi-cliente accesible desde navegadores web, con sincronización de estado en tiempo real para los 5 usuarios conectados.	Ausencia de infraestructura web, gestión de conexiones concurrentes y canal de comunicación cliente-servidor para 5 personas.	Implementar una arquitectura cliente-servidor web con backend centralizado, control de sesiones por tokens y endpoints sincronizados para salas de 5 jugadores.

1.4 Diagramas BPMN

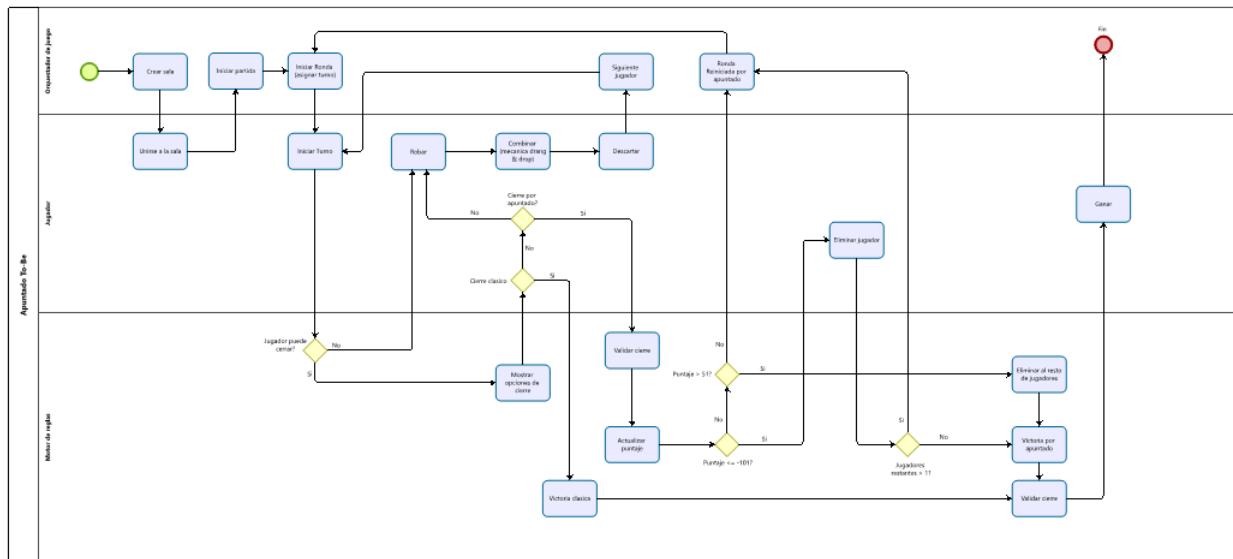
Descripción del Proceso BPMN As-Is (Modelo Manual para 5 Jugadores)

El flujo manual involucra tres carriles: **Mesa de 5 Jugadores**, **Jugador en Turno** y **Anotador de Puntos Manual**.



1. Los 5 participantes se reúnen físicamente y uno de ellos reparte las cartas de múltiples mazos.
2. El turno transcurre de forma secuencial entre los 5 puestos de la mesa (Jugador 1 al 5).
3. El jugador activo roba, organiza y descarta.
4. Si intenta cerrar la ronda, detiene el juego.
5. Los otros 4 jugadores de la mesa interrumpen sus actividades para verificar visualmente las cartas expuestas.
6. Si hay consenso, el Anotador manual calcula en su libreta las penalizaciones de los 4 perdedores y actualiza las puntuaciones de los 5 integrantes.
7. Si el jugador cierra por terna-terna-cuarta (3-3-4) y esta es validada por el anotador, finaliza la partida y el jugador que cerró gana.
8. Una compuerta exclusiva evalúa si alguno de los 5 jugadores acumuló 101 puntos negativos, en caso de que si el jugador es eliminado, después se reinicia el ciclo físico de reparto.
9. Si tras una eliminación solo queda un jugador, la partida termina, y el jugador sobrante gana.
10. Una compuerta exclusiva verifica si alguno de los jugadores acumuló mas de 51 puntos (positivos), la partida finaliza y el jugador gana.

El flujo sistematizado web se estructura en tres carriles digitales: **Clientes Web (5 Jugadores en Sala)**, **Orquestador de Juego Web (*TurnOrchestrator*)** y **Motor de Reglas y Puntaje (*HandEvaluator / RoundScoring*)**.



1. Los 5 usuarios ingresan a la aplicación web, se autentican y completan los cupos de la sala de juego configurada.
2. El **Orquestador de Juego** válida que la sala cuente exactamente con los 5 jugadores requeridos para inicializar la partida de forma automatizada y segura.
3. El sistema distribuye virtualmente las cartas: envía de forma cifrada/privada la mano correspondiente a cada uno de los 5 clientes web y publica el estado general de la mesa (mazo, pozo y marcadores).
4. El **Orquestador** bloquea o habilita las acciones basándose en el vector de turnos rotativos de los 5 jugadores.
5. Cuando el jugador en turno ejecuta una acción de cierre, la solicitud llega al servidor web, el cual delega la validación matemática al **Motor de Reglas (Strategy)**.
 - Si la validación algorítmica falla para la modalidad activa (Clásica o Apuntado < 5 pts), el sistema rechaza la solicitud notificando únicamente al cliente activo en ≤ 200 ms.
 - Si la validación es exitosa, el componente de puntaje calcula de forma automática las penalizaciones de los otros 4 jugadores restantes, actualiza el marcador global de los 5 participantes y persiste el estado en la base de datos.
6. Una compuerta exclusiva evalúa si alguno de los 5 jugadores alcanzó o superó el umbral de ≤ -101 puntos.
 - Si ningún jugador llega a -101, el sistema inicia automáticamente una nueva ronda de reparto para los 5 integrantes de la sala.
 - Si se supera el umbral, el jugador es eliminado y solo puede espectar la partida sin participar, tras eliminarse un jugador el orquestador inicia una nueva ronda empezando por el jugador siguiente al eliminado.

- Si solo queda un jugador tras una eliminación, este jugador gana automáticamente.
- 7. Una compuerta exclusiva evalúa si algún jugador superó el límite de > 51 puntos:
 - Si ningún jugador supera 51 puntos, el sistema inicia una nueva ronda de reparto para los integrantes de la sala.
 - Si algún jugador supera el límite el sistema finaliza la partida y este jugador gana automáticamente.

2. Elicitación de Requerimientos y Casos de Uso

2.1 Especificación de Requerimientos y Reglas:

2.1.1 Requisitos Funcionales.

ID	Nombre	Categoría	Descripción	Reglas	Criterio de Aceptación	Prioridad
RF 01	Crear partida	Sala/Partida	Un usuario anfitrión crea una sala eligiendo la modalidad (Clásica o Apuntado) y el límite de puntos de eliminación (por defecto -101)	RD01, RD09	Dado un usuario autenticado al crear la sala con datos válidos queda en estado "En espera". Sin límite explícito, se asigna -101.	Debe
RF 02	Unirse a sala	Sala/Partida	Un jugador se une a la sala mediante código único o enlace. El sistema impide iniciar partida hasta completar la capacidad	RD01, RD02	Dada una sala "En espera" con menos de 5 jugadores, un código válido agrega al jugador. Con 5 ocupados, un sexto se rechaza. Con menos de 5 o algún jugador desconectado, el inicio se bloquea.	Debe
RF 03	Robar carta	Ciclo de turno	El jugador en turno roba exactamente una carta, del mazo o del tope del pozo	RD03, RD04, RD10	Dado el jugador activo con 10 cartas, al robar pasa a 11 y no puede robar de nuevo. Si no es su turno, la acción se rechaza.	Debe
RF 04	Descartar carta	Ciclo de turno	El jugador activo envía una carta de su mano al pozo, lo que finaliza su turno	RD03, RD04	Dado el jugador activo con 11 cartas, al descartar queda con 10, la carta es el tope del pozo y el turno pasa al	Debe

					siguiente jugador. Descartar una carta sin haber robado, se rechaza.	
RF 05	Validar cierre Clásico	Evaluación	El sistema valida algorítmicamente si la mano del jugador que solicita el cierre cumple la estructura de la modalidad Clásica.	RD05, RD11	Dada una sala en modalidad Clásica y una mano de 11 cartas, con 1 cuarta + 2 ternas + 1 descarte válidos el cierre se acepta. En otro caso se rechaza, se notifica solo al solicitante y la partida continúa.	Debe
RF 06	Validar cierre Apuntado	Evaluación	El sistema valida algorítmicamente si la mano cumple la estructura de la modalidad Apuntado.	RD06, RD11, RD12	Dada una sala en modalidad Apuntado, con 3 ternas válidas y sobrante menor a 5 puntos el cierre se acepta. Con sobrante de 5 o más, o con menos de 3 ternas, se rechaza.	Debe
RF 07	Calcular penalizaciones	Puntaje	Tras un cierre válido, el sistema calcula los puntos en contra de los 4 jugadores restantes y actualiza el marcador acumulado.	RD07, RD12, RD13	Dado un cierre válido, cada perdedor se le resta el valor de sus cartas no combinadas, el jugador que cierra suma 10 puntos (RD13). El marcador persiste y se transmite a los 5 clientes.	Debe
RF 08	Liquidar partida	Puntaje	Tras cada ronda, el sistema evalúa el número de jugadores y el puntaje acumulado de los jugadores no eliminados para determinar el fin de la partida	RD08	Dado alguno de los escenarios planteados en la RD08, se declara correctamente al ganador.	Debe
RF 09	Eliminar Jugador	Evaluación	Luego de finalizar una ronda, el sistema verifica si hay algún jugador con puntaje menor o igual a -101 puntos, para consecuentemente eliminarlo de la partida.	RD14	Un jugador obtiene un puntaje menor o igual a -101 al cerrar una ronda, con lo cual no puede participar en la siguiente.	Debe

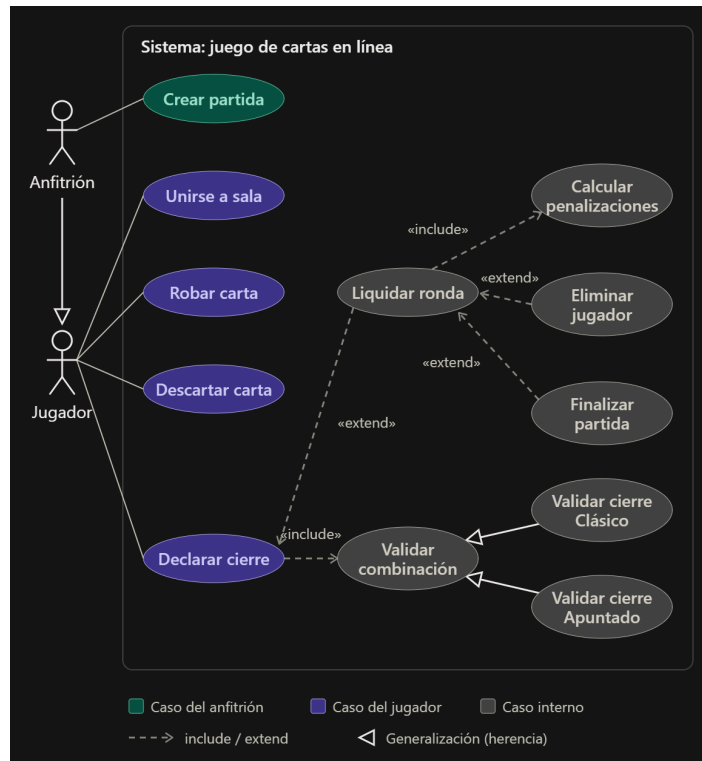
RF 10	Declarar cierre	Ciclo de turno	El jugador en turno solicita el cierre de la ronda. El sistema delega la validación a la regla de la modalidad de la sala. Si es válida cierra la ronda y la dispara para su liquidación	RD03, RD09, RD15, RD16, RD17	Dado el jugador activo con 11 cartas, al declarar cierre se valida con la modalidad de la sala. Si es válido, la ronda queda cerrada, se bloquean las acciones de los 5 jugadores y se inicia la liquidación. Si es inválido, se rechaza, se notifica solo al solicitante y su mano y su turno no cambian. Si no es su turno, si no ha robado o si la ronda ya está cerrada, se rechaza.	Debe
--------------	-----------------	----------------	--	------------------------------	--	------

2.1.2 Reglas de Dominio

ID	Regla	Enunciado	Aplica a	Estado
RD01	Capacidad de sala	Una sala tiene exactamente 5 jugadores (mínimo = máximo = 5)	RF01, RF02	Definida
RD02	Precondición de inicio	Una partida solo inicia con las 5 plazas ocupadas y los 5 jugadores conectados.	RF02	Definida
RD03	Turno activo	Solo el jugador con el turno activo puede robar, descartar o solicitar cierre. El orden del turno es en sentido antihorario.	RF03, RF04	Definida
RD04	Tamaño de mano	La mano es de 10 cartas al inicio del turno y tras descartar, y de 11 tras robar (sólo si se tiene el turno).	RF03, RF04, RF05	Definida
RD05	Estructura clásica	La mano de cierre tiene 1 cuarta, 2 ternas y 1 carta de descarte.	RF05	Definida
RD06	Estructura Apuntado	La mano de cierre tiene 3 ternas y la carta sobrante vale menos de 5 puntos. No se cuenta la carta de descarte.	RF06	Definida
RD07	Penalización	Cada carta no combinada de un perdedor suma su valor al acumulado.	RF07	Definida

RDO 8	Condición de parada	La partida termina cuando sólo queda un jugador o cuando un jugador tiene 51 puntos o más.	RF08	Definida
RDO 9	Modalidades	La modalidad es Clásica o Apuntado y no cambia una vez iniciada la partida.	RF01, RF05, RF06	Definida
RD10	Robo único	En cada turno se roba exactamente una carta, antes de descartar.	RF03	Definida
RD11	Combinaciones válidas	Una terna consiste en tres cartas del mismo valor, o una escalera de la misma pinta. Una cuarta consiste en cuatro cartas del mismo valor o una escalera de la misma pinta.	RF05, RF06	Definida
RD12	Valor de cartas	Cada carta suma su respectivo valor. Cartas con letra (A, J, Q, K) restan 10 puntos. El comodín no suma o resta puntos (0)	RF06, RF07	Definida
RD13	Puntaje de quien cierra	Al jugador que cierra se le suman 10 puntos.	RF07	Definida
RD14	Eliminación de jugadores	Jugadores con puntajes mayores o iguales a 101 puntos son eliminados de la partida.	RF09	Definida
RD15	Momento del cierre	El cierre solo puede declararse durante el turno activo, después de robar, con la mano en 11 cartas	RF10, RF05, RF06	Definida
RD16	Cierre inválido	Un cierre rechazado no modifica la mano, el turno ni el marcador, y el jugador continúa su turno.	RF10	Definida
RD17	Cierre válido	Un cierre aceptado finaliza la ronda de inmediato. Ningún jugador puede robar, descartar ni declarar cierre hasta que la ronda se liquide	RF10, RF07	Definida

2.2 Diagrama UML de Casos de Uso:



2.2.1 Trazabilidad.

Caso de uso	Actor	RF	Observación
Crear partida	Anfitrión	RF01	
Unirse a sala	Jugador	RF02	
Robar carta	Jugador	RF03	
Descartar carta	Jugador	RF04	
Declarar cierre	Jugador	RF10	
Validar Combinación	Incluido	RF05, RF06	
Validar cierre Clásico/Apuntado	Incluido	RF05/RF06	
Liquidar Ronda	Interno	Ninguno Explícito	Implícito en RF07
Calcular penalizaciones	Interno	RF07	
Eliminar jugador	Interno	RF09	

Finalizar partida	Interno	RF08	
-------------------	---------	------	--

2.3 Fichas Descriptivas de Casos de Uso Críticos.

CU1. Unirse a sala.

Requisito / Reglas	RF02 / RD01, RD02
Actor principal	Jugador
Actores secundarios	Ninguno
Precondiciones	1. Existe una sala en estado «En espera». 2. El usuario tiene el código único o el enlace de la sala.
Flujo principal	1. El jugador ingresa el código o abre el enlace de la sala. 2. El sistema verifica que la sala existe y está en estado «En espera». 3. El sistema verifica que hay cupo (menos de 5 jugadores, RD01). 4. El sistema registra al jugador como participante de la sala. 5. El sistema notifica la nueva ocupación a los clientes de la sala. 6. Si quedan las 5 plazas ocupadas y los 5 jugadores conectados (RD02), el sistema inicia la partida automáticamente: baraja, reparte las manos de forma privada y publica el estado de la mesa.
Flujos alternativos	A1. Código inválido o sala inexistente (paso 2): el sistema rechaza la solicitud e informa al solicitante. No hay cambios. A2. Sala llena (paso 3): con 5 plazas ocupadas, el sistema rechaza al sexto jugador (RD01). No hay cambios. A3. Sala ya iniciada o finalizada (paso 2): el sistema rechaza la solicitud. A4. Jugador ya registrado en la sala (paso 4): el sistema no duplica el registro. Este flujo es inferido. A5. Plazas completas pero algún jugador desconectado (paso 6): el sistema bloquea el inicio hasta que los 5 estén conectados (RD02).
Postcondiciones	Éxito: el jugador queda registrado y la sala tiene un participante más. Si se cumplió el paso 6, la partida está en curso con las manos repartidas. Fallo: el estado de la sala no cambia.

CU2. Declarar cierre.

Requisito / Reglas	RF10, RF05, RF06 / RD03, RD09, RD11, RD12, RD15, RD16, RD17
Actor principal	Jugador (con el turno activo)
Actores secundarios	Ninguno

Relaciones	«include» Validar combinación. Punto de extensión «cierre válido»: lo extiende Liquidar ronda.
Precondiciones	1. La partida está en curso y la ronda está activa (no cerrada). 2. El jugador tiene el turno activo (RD03). 3. El jugador ya robó y su mano tiene 11 cartas (RD04, RD16).
Flujo principal	1. El jugador solicita el cierre de la ronda. 2. El sistema verifica que es su turno, que la mano tiene 11 cartas y que la ronda no está cerrada. 3. El sistema obtiene la modalidad de la sala (RD09) y ejecuta «include» Validar combinación con la estrategia correspondiente: • Clásico (RF05): 1 cuarta, 2 ternas y 1 carta de descarte. • Apuntado (RF06): 3 ternas y una carta sobrante de menos de 5 puntos. 4. La validación es positiva: el sistema marca la ronda como cerrada y bloquea las acciones de todos los jugadores (RD18). 5. En el punto de extensión «cierre válido», se activa Liquidar ronda.
Flujos alternativos	A1. No es su turno o la ronda ya está cerrada (paso 2): el sistema rechaza la solicitud y notifica solo al solicitante. No hay cambios. A2. Mano distinta de 11 cartas, por ejemplo si no ha robado (paso 2): el sistema rechaza la solicitud. A3. Combinación inválida (paso 3): el sistema rechaza el cierre, notifica solo al solicitante y no modifica mano, turno ni marcador (RD17). El jugador continúa su turno. [Por definir]: penalización por cierre falso y posibilidad de reintento. A4. Solicitud duplicada (paso 2): si el jugador reenvía el cierre mientras se procesa el primero, el sistema rechaza el segundo porque la ronda ya está cerrada (RD18). A5. Desconexión del jugador durante la validación: [Por definir]. No hay RF de reconexión.
Postcondiciones	Éxito: la ronda queda cerrada, ningún jugador puede robar, descartar ni declarar cierre, y la liquidación está iniciada. Fallo: mano, turno, mazo, pozo y marcador permanecen sin cambios.

CU3. Liquidar ronda.

Requisito / Reglas	RF07, RF08, RF09 / RD07, RD08, RD12, RD13, RD14, RD15
Actor principal	Ninguno externo (caso interno iniciado por el sistema)
Beneficiarios	Los jugadores de la partida
Relaciones	Extiende Declarar cierre (condición: cierre válido). «include» Calcular penalizaciones. Sus extensiones son Eliminar jugador [puntaje $\geq$ 101] y Finalizar partida [se cumple RD08].
Precondiciones	1. Se aceptó un cierre válido y la ronda está cerrada (RD18). 2. Se conocen las manos de todos los jugadores activos.

Flujo principal	1. El sistema identifica al jugador que cerró y a los demás jugadores activos. 2. «include» Calcular penalizaciones: a cada perdedor se le resta el valor de sus cartas no combinadas (RD07). Las cartas A, J, Q y K valen 10 y el comodín vale 0 (RD12). 3. Al jugador que cerró se le suman 10 puntos (RD13). 4. El sistema actualiza el marcador acumulado y lo persiste, registrando la ronda en el historial. 5. El sistema transmite el marcador actualizado a los clientes conectados. 6. Punto de extensión «puntaje <= 101»: si algún jugador tiene -101 puntos o menos, se activa Eliminar jugador (RD15). 7. Punto de extensión «condición de parada»: el sistema evalúa RD08 con los jugadores ya eliminados. Si se cumple alguna condición, se activa Finalizar partida. 8. Si no se cumple ninguna, el sistema inicia una nueva ronda con reparto para los jugadores activos.
Flujos alternativos	A1. Uno o más jugadores obtienen puntos >= -101 (paso 6): quedan eliminados y no participan en la ronda siguiente (RD15). A2. Solo queda un jugador (paso 7): Finalizar partida lo declara ganador (RD08). A3. Un jugador llega a 51 puntos o más (paso 7): Finalizar partida lo declara ganador (RD08). A4. Se agotó el número de rondas configurado (paso 7, RD14): Finalizar partida declara ganador al de menor puntaje. A5. Fallo de persistencia (paso 4): la liquidación debe ser atómica, así que no se transmite ningún marcador parcial y se conserva el estado de ronda cerrada. Este flujo es una propuesta y debería formalizarse como requisito no funcional.
Postcondiciones	Éxito: el marcador queda actualizado y persistido, y los jugadores con 101 o más puntos quedan marcados como eliminados. Además: • Hay una nueva ronda iniciada, o • la partida está finalizada con ganador declarado. Fallo: no se emite un marcador parcial.

3. Drivers Arquitectónicos

3.1 Tabla Resumen de Drivers Arquitectónicos

ID	Driver	Tipo	Descripción y justificación	Métrica de Verificación Asociada
DA-O 1	Configurabilidad y Extensibilidad (OCP)	Atributo de Calidad	El sistema debe permitir alternar dinámicamente entre las reglas de la variante Clásica y la variante Apuntado para 5 jugadores, así como incorporar futuras variantes de juego sin modificar el código fuente principal del orquestador de rondas.	≤ 2 archivos modificados o creados en el módulo de reglas para integrar una variante de juego totalmente nueva.
DA-O 2	Testabilidad	Atributo de Calidad	Las reglas complejas de combinación de cartas, conteo y liquidación de puntajes (incluyendo los umbrales de -51 y 101 puntos)	>= 85% de cobertura de código en el núcleo de reglas y un 100%

			deben poder probarse de forma totalmente automatizada e independiente de la interfaz gráfica y de la persistencia en base de datos.	de éxito en pruebas unitarias <i>headless</i> .
DA-03	Rendimiento y Responsiveness	Atributo de Calidad	Las operaciones críticas del ciclo de juego (validación de jugadas, reparto y cálculo de puntajes para los 5 jugadores) deben resolverse de manera fluida para evitar tiempos de espera perceptibles en la interfaz de usuario.	$\leq 200ms$ de latencia máxima en la respuesta del servidor al procesar una solicitud de cierre o descarte.
DA-04	Restricción Tecnológica	Restricción	El núcleo de la aplicación de negocio debe desarrollarse utilizando un lenguaje tipado (ej. TypeScript o Java) y seguir un diseño de particionamiento modular guiado por dominio, evitando arquitecturas distribuidas innecesarias.	Uso estricto de monolito modular con separación explícita de capas lógicas sin llamadas de red internas (0 ciclos).

3.2 Análisis Detallado de los Atributos de Calidad Críticos

1. Configurabilidad y Extensibilidad (DA-01)

- **Importancia:** Las variantes de cartas (Clásica vs. Apuntado) comparten mecánicas comunes de gestión de baraja y turnos para exactamente 5 jugadores, pero difieren sustancialmente en la forma de puntuar, los cierres de ronda y las condiciones de penalización o victoria.
- **Impacto Arquitectónico:** Obliga a adoptar el patrón de diseño Strategy junto con un Registry/Factory (RuleRegistry), desacoplando por completo el algoritmo de validación de reglas de la lógica general de control de la partida.

2. Testabilidad (DA-02)

- **Importancia:** Al tratarse de un juego de lógica combinatoria estricta, la presencia de errores lógicos en el conteo de puntos o en la validación de jugadas arruina la experiencia de usuario.
- **Impacto Arquitectónico:** Demanda que los componentes del núcleo (*Domain Core*) no tengan dependencias duras hacia librerías de UI o frameworks de persistencia, permitiendo inyectar estados de prueba y ejecutar simulaciones completas de partidas en milisegundos mediante scripts de prueba automatizados.

3. Rendimiento y Responsiveness (DA-03)

- **Importancia:** Una partida de cartas requiere agilidad en el flujo interactivo. Los jugadores no deben percibir pausas al momento de descartar, robar o liquidar una ronda.
- **Impacto Arquitectónico:** Exige un diseño eficiente en memoria para las estructuras de datos que representan la baraja, las manos de los 5 jugadores y las pilas de descarte, evitando consultas innecesarias a bases de datos relacionales durante el bucle principal

de juego.

3.3 Gestión de *Trade-offs* (Compromisos Arquitectónicos)

Durante el diseño de la arquitectura para cumplir con los drivers anteriores, se asumieron los siguientes compromisos conscientes:

- **Extensibilidad vs. Complejidad Estructural Inicial:** Priorizar la configurabilidad y la extensibilidad de las reglas (DA-01) mediante abstracciones e interfaces estrictas añade una sobrecarga inicial de clases y contratos (*boilerplate*). Sin embargo, este *trade-off* está plenamente justificado, ya que aísla los cambios futuros y reduce drásticamente el costo de mantenimiento ante modificaciones en las reglas de negocio.
- **Desacoplamiento vs. Velocidad de Desarrollo Directo:** Evitar el acoplamiento entre la interfaz gráfica y el motor de reglas para garantizar la testabilidad (DA-02) impide realizar atajos de código (como validar reglas directamente en los controladores visuales), requiriendo un rigor estricto en la transferencia de datos mediante DTOs y eventos de estado.

4. Arquitectura Lógica y Descomposición Modular

4.1 Identificación de Componentes Lógicos

4.1.1 Enfoque aplicado (Workflow)

El juego sigue un proceso secuencial ya documentado en el BPMN To-Be de la sección 1.4 (reparto -> turno -> cierre -> liquidación), con fases claras y varias validaciones intermedias. Por eso se elige el enfoque *Workflow* sobre *Actor/Action*: es más fácil descomponer por flujo, y los componentes los derivamos directamente sobre los pasos ya definidos. Nuestro proceso de construcción es iterativo.

4.1.2 Mapeo Workflow → Componentes

ID	Paso (happy path)	Lane	Componente	RF
A1–A2	Crear la partida, elegir modalidad y fijar parámetros (comodín, umbral)	Jugador (anfitrión)	Match Setup	RF01
A3	Asociar la regla activa de la modalidad	Orquestador	Rule Selection	RF01
A4	Unirse a la sala	Jugador	Match Setup	RF02
B1–B3	Inicializar y barajar el mazo, repartir y abrir el pozo	Orquestador	Deck Management	—

B4	Asignar el primer turno	Orquestador	Turn Orchestration	—
B5	Enviar a cada jugador su mano privada y el estado público	Orquestador	Game Notification	—
C1	Habilitar las acciones permitidas en la fase actual (State)	Orquestador	Turn Orchestration	RF03, RF04
C2–C3	Robar del mazo o del pozo y descartar	Jugador → Orquestador	Deck Management	RF03, RF04
C4	Registrar el movimiento	Orquestador	Match History	—
C5	Notificar el cambio de estado	Orquestador	Game Notification	—
C6	Ceder el turno al siguiente jugador	Orquestador	Turn Orchestration	RF04
D1–D2	Declarar el cierre y delegarlo a la regla activa ya inyectada	Jugador → Orquestador	Turn Orchestration	RF05, RF06
D3–D4	Compuerta de cierre: Clásica (1 cuarta + 2 ternas + 1 descarte) o Apuntado (3 ternas + sobrante < 5 pts)	Motor reglas de	Game Modes	RF05, RF06
E1	Calcular el valor de las cartas sobrantes	Motor reglas de	Hand Evaluation	RF07
E2	Aplicar penalizaciones y actualizar acumulados	Motor reglas de	Round Scoring	RF07
E3	Registrar la ronda en el historial	Orquestador	Match History	—
E4	¿Puntaje ≥ 101? → eliminar jugador	Motor reglas de	Match Outcome	RF09
E5	Finalizar la partida o iniciar una ronda nueva (B1)	Orquestador	Match Outcome	RF08
E6	Notificar el resultado de la ronda o partida	Orquestador	Game Notification	—

4.2 Declaraciones de Rol y Responsabilidad

4.2.1 Declaraciones por componente

Componente	Namespace	Declaración de rol y responsabilidad	Pasos	Fuera de su alcance
Match Orchestration	apuntado.orchestration	Responsable exclusivo de coordinar la ejecución de cada comando entre los módulos del núcleo: invoca, no decide.	Todos (coordinador)	Validar jugadas, calcular puntajes, decidir turnos o el fin de la partida
Match Setup	apuntado.match.setup	Responsable exclusivo de constituir la partida antes de su inicio, dejando fijadas su configuración y su lista de jugadores.	A1–A2, A4	Elegir la regla de juego, repartir cartas, iniciar turnos
Rule Selection	apuntado.rules.selection	Responsable exclusivo de resolver, al crear la partida, qué regla de juego aplica según la modalidad seleccionada.	A3	Evaluar manos, calcular puntajes
Deck Management	apuntado.deck	Responsable exclusivo de administrar las reservas de cartas de la ronda, garantizando que cada carta esté en un único lugar.	B1–B3, C2–C3	Decidir si un jugador puede robar o descartar, validar combinaciones
Turn Orchestration	apuntado.turn.orchestration	Responsable exclusivo de coordinar la secuencia de turnos y fases de la ronda, decidiendo qué acción está permitida para cada jugador en cada momento.	B4, C1, C6, D1–D2	Validar combinaciones, calcular puntos, mover cartas directamente
Game Modes	apuntado.rules.modes	Responsable exclusivo de encapsular las condiciones de victoria de cada modalidad detrás del contrato GameRule.	D3–D4	Reconocer combinaciones, valorar cartas, elegir la modalidad, acumular puntajes, secuenciar turnos
Hand Evaluation	apuntado.rules.evaluation	Responsable exclusivo de calcular la mejor descomposición de un conjunto de cartas en combinaciones (ternas y cuartas) según el patrón recibido, incluido el valor del sobrante.	E1	Conocer la modalidad o la regla activa, aplicar penalizaciones, decidir turnos
Round Scoring	apuntado.scoring.round	Responsable exclusivo de llevar el acumulado de puntos de cada jugador a partir de las penalizaciones de cada ronda liquidada.	E2	Calcular el valor de las cartas (lo define la regla activa), validar el cierre, decidir eliminaciones o el fin de la partida
Match Outcome	apuntado.match.outcome	Responsable exclusivo de determinar, tras cada liquidación, si la partida continúa o finaliza según la modalidad.	E4–E5	Calcular puntos, reiniciar la baraja

Game Notification	apuntado.notification	Responsable exclusivo de difundir los cambios de estado a los jugadores, entregando a cada uno solo la información que le corresponde.	B5, C5, E6	Decidir o modificar el estado del juego
Match History	apuntado.match.history	Responsable exclusivo de registrar cronológicamente las acciones y rondas de la partida para consulta y auditoría.	C4, E3	Calcular puntajes, validar jugadas, definir el formato de los eventos

4.2.2 Justificación por características arquitectónicas

Componente	Característica que justifica aislarlo
Rule Selection	Mantenibilidad (DIP): Registry/Factory. Es el único punto que conoce las implementaciones concretas. Match Setup, Turn Orchestration y Match Outcome dependen solo de la clase abstracta GameRule.
Game Modes	Extensibilidad (OCP): Strategy + Template Method. <code>validate_closing()</code> fija el algoritmo y cada modalidad implementa solo <code>_closing_pattern()</code> y <code>_accepts_leftover()</code> . Una modalidad nueva se agrega con un archivo en <code>res://rules/modes/</code> , un valor en el catálogo GameMode y un <code>register()</code> , sin tocar a ningún consumidor (RNF 1).
Hand Evaluation	Testability + Responsiveness: lógica pura, sin estado ni árbol de escena, que se prueba headless y se mide aislada contra los ≤ 150 ms del RNF 2.
Game Notification	Security + Usabilidad: filtra lo que ve cada jugador (mano privada) y desacopla la UI mediante Observer/signals (RNF 3 y 6).
Match History	Auditabilidad + Reliability: el log de movimientos permite resolver disputas (RNF 5) y reconstruir el estado al reconectar (RNF 4)
Round Scoring y Match Outcome	Configurabilidad (SRP): cambian por razones distintas. Las penalizaciones varían con apuestas; la condición de fin varía con el umbral 101 o el reingreso.

4.2.3 Verificación contra el Entity Trap

Ningún componente representa una entidad del dominio. No existen GameManager, CardManager ni PlayerManager: los componentes se derivan de los pasos del flujo (4.1.3) y sus nombres describen una acción (Setup, Evaluation, Scoring, Notification). Carta, Mano y Jugador aparecen como clases del modelo de dominio (4.4), no como componentes. Cada declaración de 4.2.1 tiene una sola responsabilidad y un "fuera de su alcance" explícito.

4.3 Diagrama UML de Componentes Lógicos

4.3.1 Diagrama

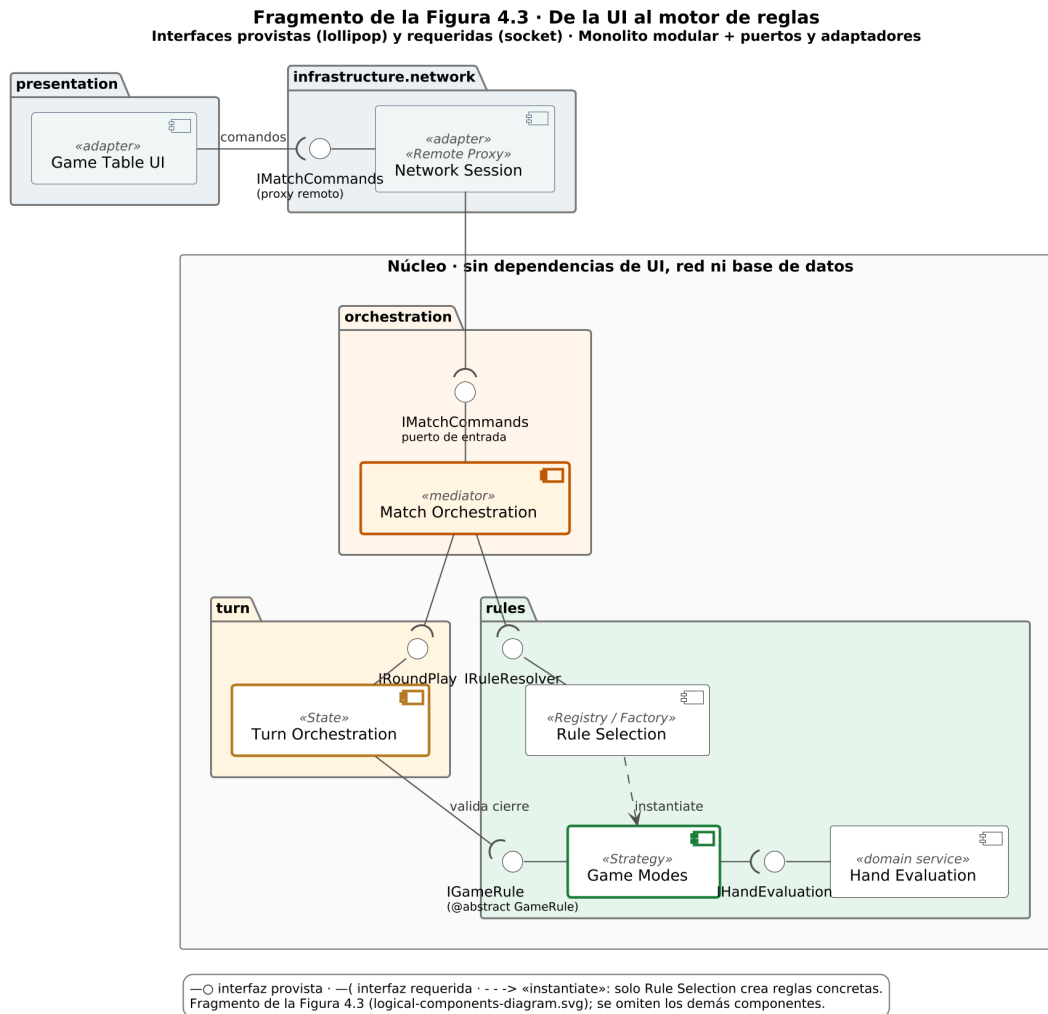


Figura 4.3. Fragmento del diagrama de componentes lógicos: de la UI al motor de reglas.

Diagrama completo:

<https://github.com/Thom204/ApuntadoFdA/blob/main/docs/diagrams/logical-components-diagram.svg>.

4.3.2 Lectura del diagrama (puertos y adaptadores, estereotipos)

Puertos y adaptadores. El núcleo no depende de la UI, la red ni la base de datos. Game Table UI solo requiere `IMatchCommands`, que Network Session provee como *Remote Proxy*: reenvía el comando por RPC, y en el servidor valida la sesión antes de delegar (RNF 6). Match Orchestration expone ese mismo contrato como puerto de entrada. Las dependencias siempre apuntan hacia el núcleo. Por eso Game Table UI tiene $Ca = 0$ (4.5.3): rediseñar la interfaz o tematizar las cartas no afecta a ningún otro componente.

4.4 Diagrama UML de Clases de Dominio

4.4.1 Diagrama

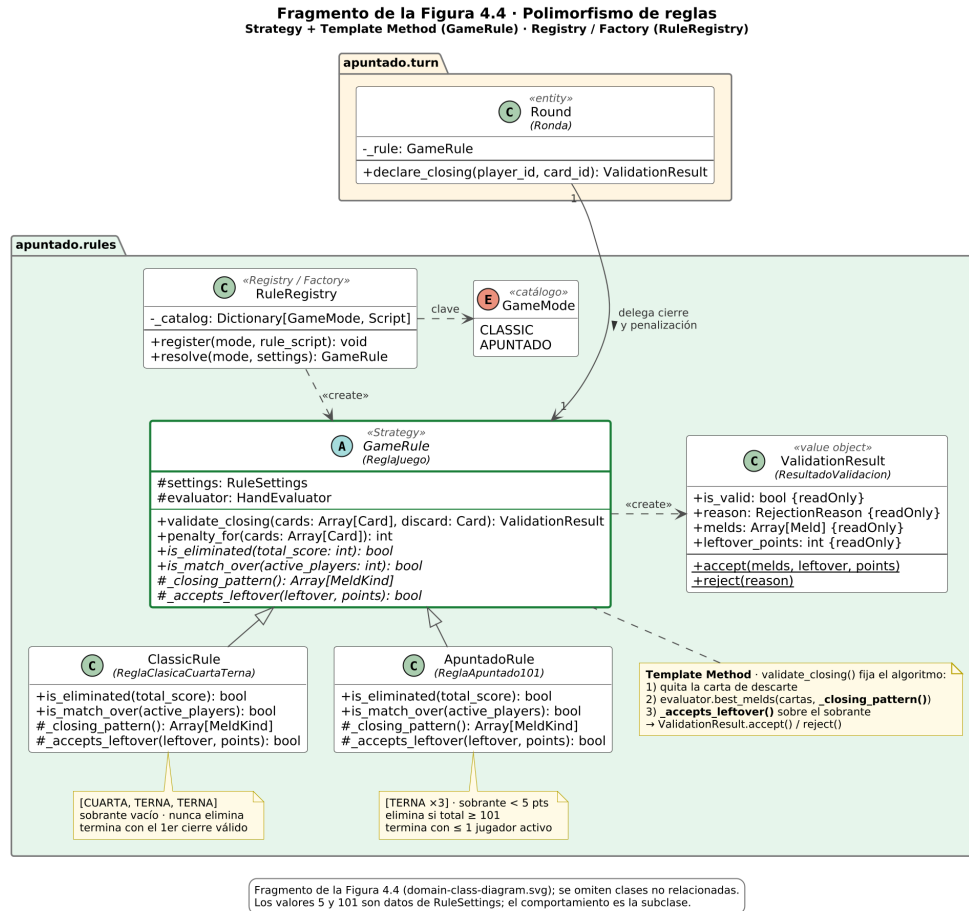


Figura 4.4 Fragmento del diagrama de clases: polimorfismo de reglas. Diagrama completo:

<https://github.com/Thom204/ApuntadoFdA/blob/main/docs/diagrams/domain-class-diagram.svg>

4.4.2 Patrones de diseño e invariantes reflejados en el modelo

Strategy + Template Method (GameRule). Round delega el cierre a la clase abstracta GameRule sin conocer la modalidad (DIP). Su método validate_closing() es un Template Method con tres pasos fijos: separa el descarte, busca combinaciones y evalúa el sobrante. ClassicRule y ApuntadoRule sólo implementan lo que varía: el patrón de cierre ([CUARTA, TERNA, TERNA] frente a [TERNA × 3]), la aceptación del sobrante (vacío frente a < 5 puntos) y las condiciones de eliminación y fin de partida. Los valores 5 y 101 son datos de RuleSettings, así que solo un comportamiento nuevo justifica una subclase. La regla recibe una copia de las cartas, nunca la Hand (H4).

Registry / Factory (RuleRegistry). Es la única clase que conoce las reglas concretas. Las instancia según GameMode, lo que elimina los condicionales por modalidad del núcleo

(OCP). Agregar una modalidad requiere un archivo nuevo y modificar solo dos existentes (RNF 1).

State (TurnPhase). Turn delega en su fase actual (DrawPhase → DiscardPhase → EndPhase), y Round consulta Turn.can() antes de cada acción. Así se garantizan por estructura el turno activo (RD03) y el robo único antes de descartar (RD10). State modela *cuándo* se actúa y cambia con cada acción. Strategy modela *qué* gana y queda fija durante la partida (RD09).

Observer (señales). Round y Match emiten eventos (move_performed, round_closed, match_finished...) a los que se suscriben Match History, Game Notification y la UI. El emisor no conoce a sus suscriptores (OCP, H2).

Value Objects e invariante de conservación. Card, Meld y ValidationResult son inmutables, por lo que cruzan módulos sin copias defensivas (RNF 6). Card no guarda puntos: su valor sale de CardValueTable. Cada carta, identificada por un instance_id único, está en un solo lugar (Deck, DiscardPile o una Hand). Round es el único dueño de esas zonas, las cartas se trasladan con take → add/push y nunca se copian, y el cliente envía card_id, no objetos Card.

4.5 Evaluación de Acoplamiento y Ley de Deméter

Esta evaluación cierra el ciclo de identificación y reestructuración de componentes descrito en 4.1.1 (Cap. 8, Fig. 8-6). Revisa cuánto sabe cada componente de los demás (Ley de Deméter) y mide el acoplamiento aferente (Ca, dependencias entrantes), el referente (Ce, dependencias salientes) y la Inestabilidad $I = Ce / (Ca + Ce)$ (Cap. 3, Ec. 3-4).

4.5.1 Aplicación de la Ley de Deméter

Cuando un jugador declara el cierre (paso D1), Turn Orchestration envía un único mensaje a la regla activa y recibe un value object inmutable:

```
# ✗ Round atraviesa la regla para usar su evaluador y fija la modalidad en el turno
_rule._evaluator.best_melds(cards, [CUARTA, TERNA, TERNA], _rule._settings.card_values)

# ✓ Round solo conoce el contrato IGameRule
var result: ValidationResult = _rule.validate_closing(hand.cards(), discard)
```

El mismo principio aplica en el resto del diseño:

Interacción	Lo que el cliente pide	Lo que no necesita conocer	Mecanismo
Game Table UI ->	Robar, descartar	Validez de la jugada, turno,	Remote Proxy +

IMatchCommands	card_id, cerrar	red, dominio	Facade
Match Orchestration -> Round	draw_from_deck(player_id)	Deck, DiscardPile, Hand; cuándo recargar el mazo	<i>Tell, don't ask</i> ; copias defensivas
Match Orchestration -> Match Outcome	conclude_round()	Penalizar, recalcular acumulados, evaluar eliminación y fin	Raíz de agregado
Match Orchestration -> Rule Selection	resolve(mode, settings)	ClassicRule y ApuntadoRule	Registry/Factory (DIP)
Núcleo -> Notificación, Persistencia y Red	Emitir eventos, usar puertos	Quién escucha, motor de BD, transporte	Observer + puertos y adaptadores

La fila de Match Outcome aplica el caso de las Fig. 8-14 y 8-15 del libro. El conocimiento de la liquidación pasó del orquestador a la raíz de agregado Match, así que el orquestador no depende de Round Scoring. Como advierte el libro, esto redistribuye el acoplamiento en lugar de eliminarlo: Match Outcome lo absorbe (Ce = 5). Se acepta porque Match es el dueño natural de la consistencia de la partida.

Entre módulos solo cruza connascence de nombre y de tipo: firmas, class_name y value objects cómo Card o ValidationResult. Las formas fuertes (el algoritmo de combinaciones, el orden de las fases, el orden de liquidación) quedan encerradas dentro de un solo componente.

4.5.2 Hallazgos y ajustes al modelo (H1–H5)

Al conciliar los diagramas 4.3 y 4.4 aparecieron dos ciclos, que el Cap. 6 señala como el antipatrón más dañino para la modularidad, y cuatro puntos donde un componente sabía más de lo necesario. Todos los ajustes ya están incorporados:

- **H1:** Round se ubica en res://turn/, para que 4.3 y 4.4 coincidan.
- **H2:** MoveRecord pasa a res://turn/ como carga útil de RoundEvents, lo que rompe el ciclo Turn Orchestration ↔ Match History.
- **H3:** CardValueTable pasa a res://rules/evaluation/, lo que rompe el ciclo Game Modes ↔ Hand Evaluation.
- **H4:** GameRule recibe Array[Card] y no la entidad Hand, así que la regla no puede alterar la mano.
- **H5:** GameMode se declara como catálogo propio en res://rules/, y Match Setup deja de conocer RuleRegistry.
- **H6:** la validación del cierre (D2) se asigna a Turn Orchestration. Rule Selection solo interviene en A3, porque la regla se inyecta al crear la partida.

4.5.3 Matriz de dependencias y métricas de acoplamiento (Ca, Ce, I)

Criterio de conteo: una dependencia de A hacia B se cuenta una vez cuando A requiere una interfaz de B, usa un tipo de B o escucha una señal de B. El emisor de una señal no depende de sus suscriptores, y un adaptador que implementa un puerto del núcleo depende del núcleo, no al revés. Todas las dependencias aparecen en el diagrama completo (Anexo A); las que son solo de tipo se dibujan con flecha punteada «tipo».

Componente	Ca	Ce	I	Lectura
Match Orchestration	3	6	0,67	Mediator: concentra la coordinación en un solo punto
Match Setup	2	1	0,33	Solo conoce el contrato de reglas (RuleSettings, GameMode)
Match Outcome	1	5	0,83	Absorbe la liquidación diferida
Match History	2	1	0,33	Solo conoce el evento de turno (MoveRecord)
Turn Orchestration	3	3	0,50	Depende del contrato IGameRule, nunca de una regla concreta
Round Scoring	2	0	0,00	Estable y concreto
Rule Selection	1	2	0,67	Único componente que conoce las reglas concretas
Game Modes (IGameRule)	5	2	0,29	El más usado; se depende solo de @abstract GameRule
Hand Evaluation	2	1	0,33	Solo conoce Card
Deck Management	3	0	0,00	Base estable; su carga entrante viene de Card, que es inmutable
Game Notification	1	1	0,50	Observer: solo escucha IGameEvents
ADAPTADORES	–	–	–	–
Game Table UI	0	2	1,00	Nada depende de él, así que rediseñar la interfaz no afecta a ningún otro componente.
Card Theming	1	0	0,00	Proveedor de Resources de tema. Cambiar el tema cambia Resources
Network Session	1	2	0,67	Remote Proxy: primario (IMatchCommands) y secundario (IPlayerChannel)
Match Persistence	0	1	1,00	Adaptador secundario (driven)

Adaptadores. Game Table UI, Card Theming, Network Session y Match Persistence dependen del núcleo, y ningún componente del núcleo depende de ellos. Game Table UI y Match Persistence tienen $Ca = 0$, así que la tematización o un cambio de base de datos no afectan al núcleo.

Resumen. 15 componentes, 27 dependencias y 0 ciclos. La tabla es consistente: $\Sigma Ca = \Sigma Ce = 27$, porque cada dependencia se cuenta una vez como saliente en su origen y una vez como entrante en su destino.

5. Decisiones Arquitectónicas Iniciales (ADRs)

5.1 ADR-01: Particionar el sistema por dominio del juego mediante un monolito modular

ESTADO: Accepted

CONTEXTO: El sistema debe soportar dos modalidades de juego (Clásica 4-3-3 y Apuntado 101) y evolucionar con variantes que el cliente ya anticipó: comodín, apuestas dinámicas, reingreso de eliminados y variaciones visuales. El driver principal es la Extensibilidad/Configurabilidad, seguido de Testability y Responsiveness.

Se evaluaron dos alternativas de particionamiento superior:

- Particionamiento técnico (capas de presentación, negocio y persistencia). Es simple y conocido, pero un cambio de regla atraviesa todas las capas.
- Particionamiento por dominio (módulos alineados con las responsabilidades del juego). Cada módulo agrupa todo lo necesario para una parte del juego, y los componentes identificados ya siguen esta forma.

DECISIÓN: Usaremos particionamiento por dominio como estructura superior, implementado como un monolito modular. Cada módulo corresponde a un área del juego y se organiza como una carpeta de nivel superior en el árbol de proyecto (res://match/, res://turn/, res://rules/, etc.), replicando el namespace lógico del inventario de 4.1. Los módulos exponen su funcionalidad a los demás solo mediante clases con `class_name` y señales (signals); ningún módulo accede a nodos internos de otro por ruta directa.

Justificación: Los cambios que el cliente anticipa son cambios de dominio, no de tecnología. Un cambio de regla queda contenido en el módulo de reglas. Se elige un monolito y no servicios distribuidos porque ninguna característica exige despliegue independiente o escalabilidad por partes. **CONSECUENCIAS:**

- *Positivos:* los cambios de reglas quedan localizados en un módulo. Los módulos se pueden probar de forma aislada. Si en el futuro se necesita distribuir, los módulos ya tienen fronteras claras para extraerse como servicios.
- *Negativos:* las funciones transversales (notificación, historial) son usadas por varios módulos y deben diseñarse con cuidado para no acoplarlos entre sí. Las fronteras

entre módulos dependen de la disciplina del equipo de desarrollo porque en un monolito nada impide técnicamente que un módulo acceda a las clases internas de otro. Todo el sistema se despliega y escala como una unidad.

CUMPLIMIENTO (COMPLIANCE):

- *Automated fitness function: un script de dependencias ejecutado en CI (GUT, modo headless) construye el grafo de referencias entre carpetas. Falla si: aparece un ciclo entre módulos; un archivo del núcleo referencia `res://presentation/` o `res://infrastructure/`; un módulo accede a otro por ruta de nodo en lugar de su API pública.*
- *Manual: en la revisión de código se rechazan las cadenas de llamadas sobre objetos de otro módulo (Ley de Deméter) y cualquier lógica de negocio agregada a MatchOrchestrator.*

NOTAS: Daniel Restrepo, 09-26-2026, revisores, Anexo B (0001-particionamiento-por-dominio.md).

5.2 ADR-02: Implementar las reglas de victoria como estrategias intercambiables dentro del monolito modular

ESTADO: Accepted

CONTEXTO: Las modalidades Clásica (1 cuarta + 2 ternas + 1 descarte) y *Apuntado* (3 ternas con sobrante menor a 5 puntos y eliminación en 101) tienen condiciones de victoria distintas, y el cliente anticipa nuevas variantes. Los drivers de Extensibilidad y Testability exigen agregar reglas sin modificar el núcleo y probarlas de forma automatizada, y el RNF 1 fija un máximo de 2 clases modificadas por regla nueva. La variación de parámetros se resuelve con configuración; esta decisión trata la variación de comportamiento. Se evaluaron tres alternativas: condicionales por modalidad en el núcleo, Strategy dentro del monolito modular (adoptada) y Microkernel con plugins cargados en ejecución.

DECISIÓN: Las reglas de victoria se implementarán con el patrón Strategy dentro del módulo de reglas del monolito modular. *ClassicRule* y *ApuntadoRule* son las estrategias concretas, ubicadas en `res://rules/modes/` y apoyadas en Hand Evaluation. *RuleRegistry* selecciona la estrategia según la modalidad de la partida y es el único punto que conoce las implementaciones concretas. Turn Orchestration depende solo de *GameRule*. Los parámetros de regla residen en *RuleSettings*, que *MatchConfig* compone.

Justificación: una modalidad nueva equivale a un archivo en `res://rules/modes/`, un valor en el catálogo *GameMode* y una línea en el registro, sin tocar el núcleo, la interfaz ni la persistencia. Las reglas se prueban sin árbol de escena.

CONSECUENCIAS:

- *Positivas:* modalidades agregables sin modificar el núcleo, reglas probadas de forma

aislada, eliminación de la ramificación por modalidad y polimorfismo explícito en el diagrama de clases.

- *Negativas*: cada modalidad nueva requiere volver a exportar el juego, se agrega una capa de abstracción. Agregar una modalidad modifica dos archivos existentes (GameMode y RuleRegistry), el máximo que permite el RNF 1.

CUMPLIMIENTO (COMPLIANCE):

- *Automatizado*: pruebas de contrato con GUT, ejecutadas en modo headless, que aplican a una batería común de casos a todas las reglas registradas.
- *Manual*: en la revisión de cada regla nueva se verifica el límite del RNF 1 (máximo 2 archivos existentes modificados).

NOTAS: Daniel Restrepo, 09-26-2026, revisores, Relacionado con: ADR-01, secciones 4.4 y 4.5, RNF 1 y RNF 6. Anexo B (0002-reglas-como-estrategias.md).